INTERNATIONAL UNIVERSITY

DEPARTMENT OF PHYSICS

REPORT LAB01
LIBRARY RESOURCE MANAGEMENT SYSTEM

Students 's Name:

Duong Ngoc Anh Thu - SESEIU24026

1. Class Identification and Responsibilities (Task 1)

Based on the source code (Library.swift), the system is designed with the following classes to represent library resources and handle related operations, ensuring encapsulation and extensibility according to Object-Oriented Programming (OOP) principles:

- **LibraryResource:** The base class represents all library resources. It is responsible for storing common attributes (ID, title, base fee, availability status) and defining core behaviors (request access, return) applicable to all resources.
- **PrintedBook:** A subclass inheriting from LibraryResource, managing specific information for printed books, including the author and the standard identifier (ISBN).
- **AcademicJournal:** A subclass managing academic journals. It handles the volume, issue number, and specifically a canBorrow flag to determine if the journal can be taken home or is for in-library reference only.
- **DigitalMaterial:** A subclass managing digital resources, responsible for storing platform information and controlling the number of concurrent users (currentUsers / maxUsers).
- **LibraryError:** An Enum conforming to the LocalizedError protocol, used to define and classify errors during invalid operations (e.g., resource already borrowed, non-borrowable journal, max concurrent users reached).

2. Attributes and Methods (Task 2)

Attributes and methods are clearly separated between shared properties and resource-specific properties:

2.1. LibraryResource Class (Shared Information)

- **Attributes:** id (UUID), title (String), baseFee (Double), isAvailable (Bool). These are mandatory attributes for all resource types to enable system identification and UI display.
- **Methods:** requestAccess() throws -> Double (executes the borrow/access operation and returns the fee, throwing an error if unavailable), returnResource() (restores availability status).

2.2. PrintedBook Class (Specific Information)

- **Attributes:** author (String), isbn (String).

2.3. AcademicJournal Class (Specific Information)

- **Attributes:** volume (Int), issue (Int), canBorrow (Bool).

2.4. DigitalMaterial Class (Specific Information)

- **Attributes:** platform (String), maxUsers (Int), currentUsers (Int).

3. Inheritance and Polymorphism (Task 3)

The design applies Inheritance and Polymorphism to meet the system's flexibility and extensibility requirements.

3.1. Inheritance Design

The three classes: PrintedBook, AcademicJournal, and DigitalMaterial all inherit directly from LibraryResource. This provides a significant advantage for the View Model (LibraryViewModel), allowing it to manage a unified array [LibraryResource] instead of creating multiple separate arrays for each resource type. If the library needs to add a new resource type in the future, the system only needs to create a new subclass inheriting from LibraryResource without breaking the existing architecture.

3.2. Polymorphic Behavior via requestAccess() Method

The requestAccess() method is defined generally in the base class, but each subclass can override it to implement its specific validation logic:

- **AcademicJournal:** Overrides requestAccess() to check the canBorrow flag. If this flag is false (reference only), the method rejects the operation by throwing a LibraryError.notBorrowable error before calling the base class method (super.requestAccess()).
- **DigitalMaterial:** Overrides requestAccess() to compare currentUsers and maxUsers. If the user count reaches the limit, it throws a LibraryError.maxUsersReached error. Otherwise, it increments the user count, and if all slots are filled, it proactively changes the isAvailable status to false.

4. Encapsulation and Validation (Task 4)

Encapsulation is strictly enforced to protect the internal state of objects from unauthorized external modifications, ensuring data validity.

- **Using fileprivate(set) Access Control:** In the LibraryResource class, the isAvailable attribute is declared as @Published fileprivate(set). External Views (like SwiftUI Views) only have read access to this state for display purposes ("Available" / "Unavailable") and cannot directly assign a value.
- **State Modification via Valid Methods:** Changing the availability status is only permitted indirectly through the requestAccess() and returnResource() methods. Within these methods, the system applies guard statements for strict condition checking (e.g., is it already borrowed, is it allowed to be borrowed, are there available access slots).
- **Managing currentUsers in DigitalMaterial:** Similarly, the concurrent user counter (currentUsers) is also locked by fileprivate(set). Any operation to increment or decrement the user count occurs entirely within the internal borrow and return methods of this class.